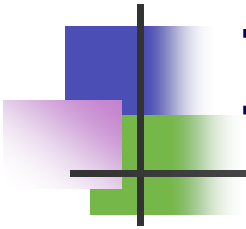


Arquitetura de Software

Aula 8

Framework backend Spring. Spring Boot.

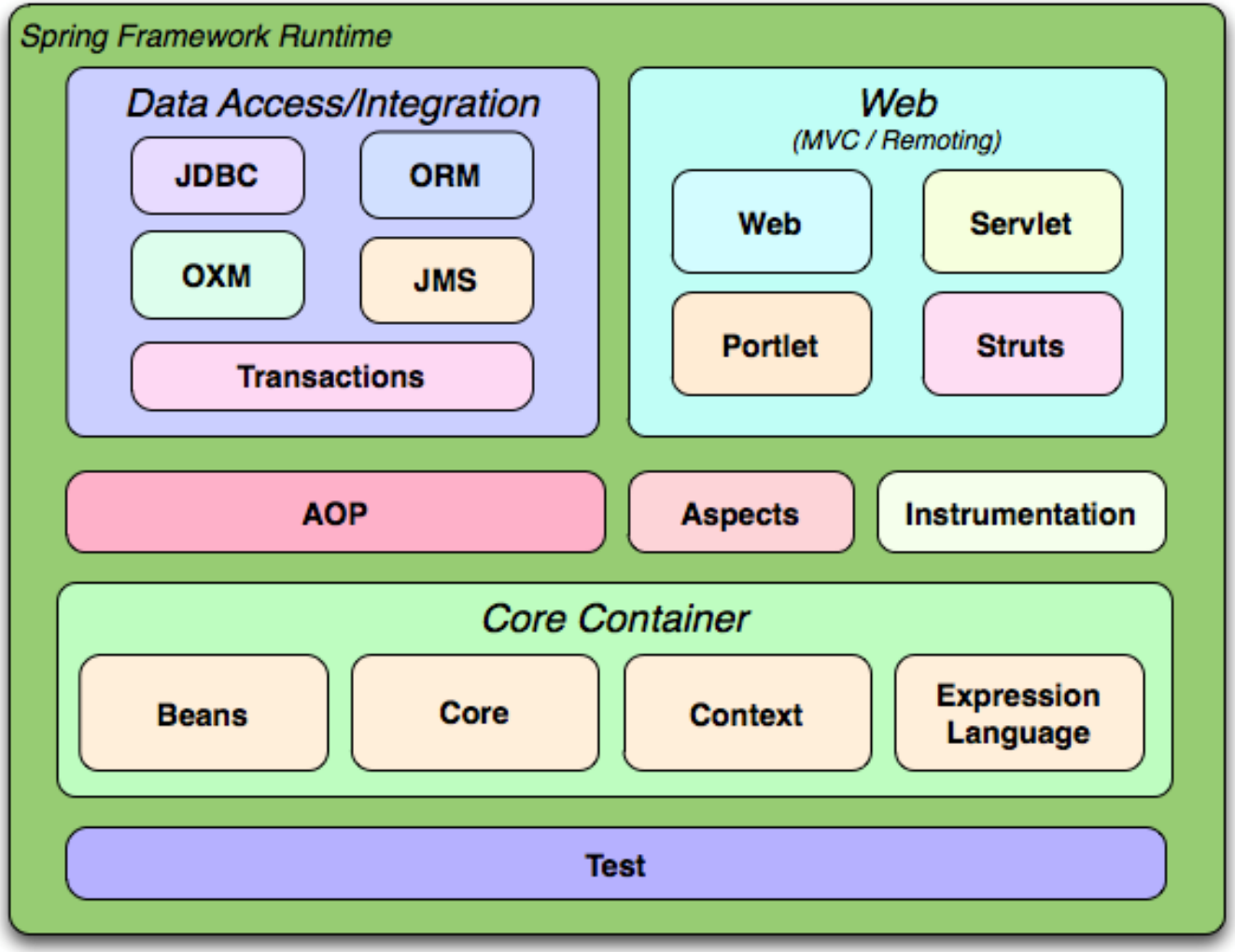
Prof. José Martins Junior



Introdução

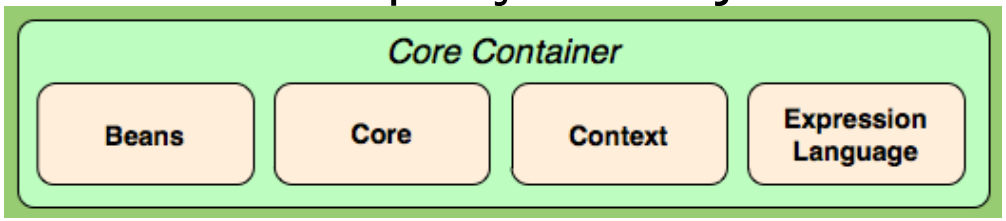
- Spring framework
  - Plataforma Java que oferece suporte à infraestrutura para desenvolvimento de aplicações
- Tipos e aplicações
  - Web
  - Corporativas
  - Standalone
  - Em lotes
  - Integração

Módulos Spring



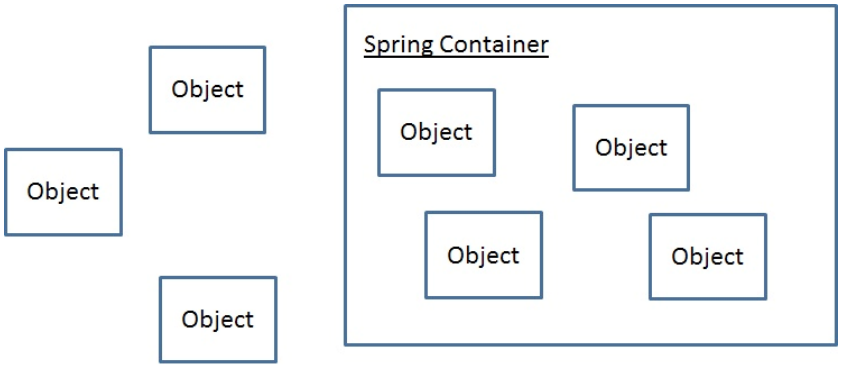
Módulos Spring

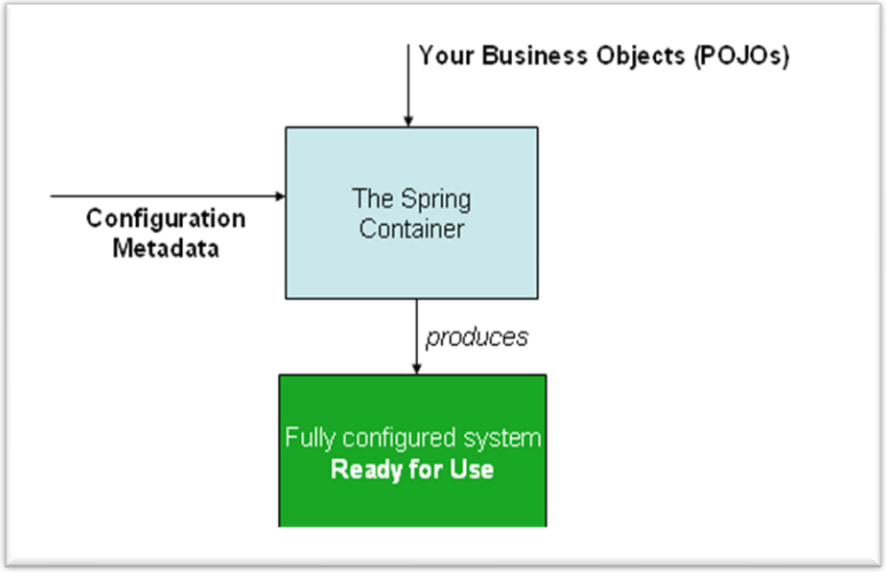
- Core e Beans são partes essenciais do framework
  - Inversão de Controle
  - Injeção de dependências
  - BeanFactory
- Context incrementa mais recursos aos primeiros
  - Acesso em nível de framework a objetos gerenciados
  - Suporte a contexto (e de aplicação)
  - Internacionalização, propagação de eventos e Java EE
- Expression Language
  - Suporte a busca e manipulação de objetos em tempo real

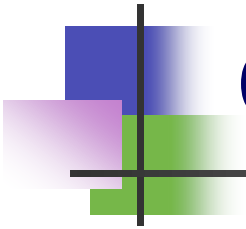


Container Spring

- O container IoC Spring é responsável por instanciar, configurar e construir beans

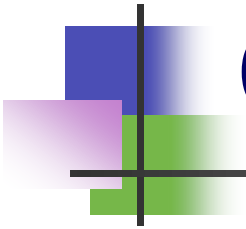






Configurações possíveis

- XML-based
  - Configuração via metadados em formato XML
  - Forma mais antiga de configuração
- Annotation-based
  - Suporte introduzido no Spring 2.5
  - Anotações em classes Java, no lugar de arquivos XML
- Java-based
  - Introduzida na versão 3.0 do Spring
  - Permite definir beans com anotações fora das classes da aplicação



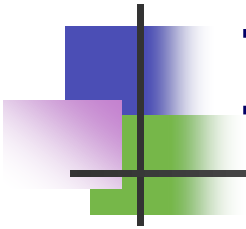
Ciclo de vida de um bean

- ID
  - Identificador único de um bean
  - Único por contexto
- Name
  - Pode ser usado para definir apelidos
  - Não tem que ser único
  - Podem ser especificados múltiplos nomes separados por vírgula, ponto e vírgula, ou espaço
- Unique name (container-generated)
  - Gerado apenas se nem id ou name forem fornecidos
  - Não pode ser ligado (wired) com elementos de referência

Escopos de beans

- Principais escopos

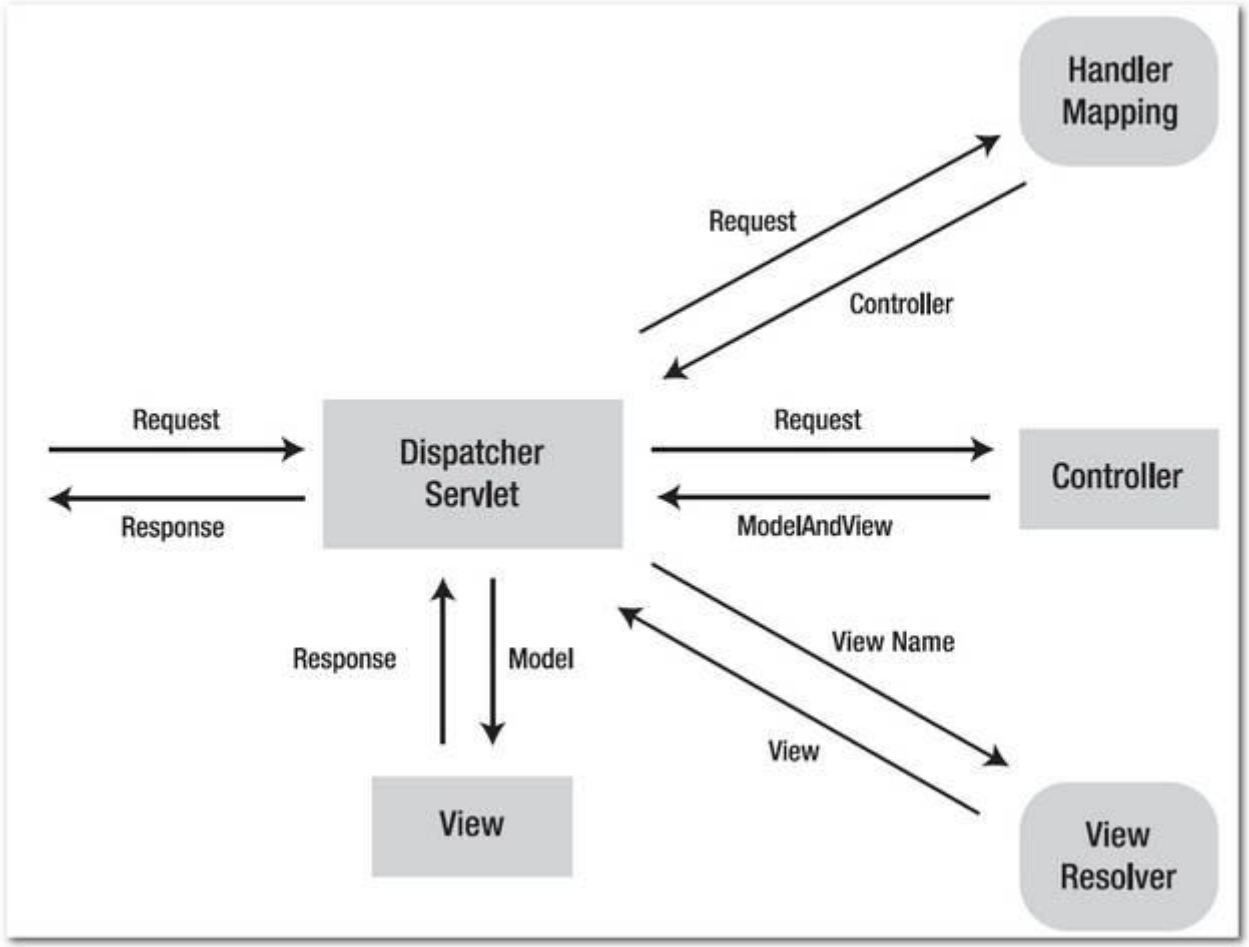
Scope	Description
singleton	This scopes the bean definition to a single instance per Spring IoC container (default).
request	This scopes a bean definition to an HTTP request.
session	This scopes a bean definition to an HTTP session.
global-session	This scopes a bean definition to a global HTTP session.

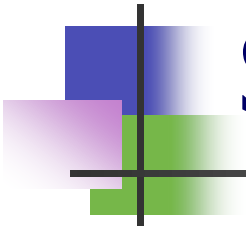


Injeção de dependências

- Tipos de anotações em classes
  - @Component - indica um componente auto scan
  - @Repository - indica componente DAO (persistência)
  - @Service - indica um componente de serviço (negócio)
  - @Controller - indica um controlador (apresentação)

Spring MVC – ciclo de vida





Spring MVC – ciclo de vida

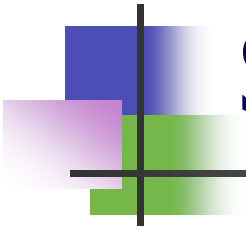
- Dispatcher servlet
  - Controlador principal do framework
  - Responsável por delegar o controle durante as fases de execução de um request HTTP
- HandlerMapping
  - Seleciona objetos que manipulam as requests
  - Baseiam-se em algum atributo ou condição interna/externa
  - Spring seleciona como padrão RequestMappingHandlerMapping

Spring MVC – ciclo de vida

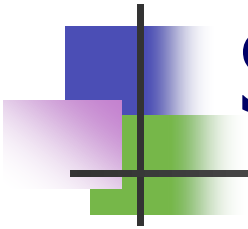
- Controller
  - Localiza-se entre o Modelo e a View (faz chaveamento entre eles)
  - Gerencia requisições e redireciona à resposta apropriada
  - Pode ser declarado com a anotação @Controller
  - Isso permite incluir controladores em POJOs ou outras classes



Spring MVC – ciclo de vida



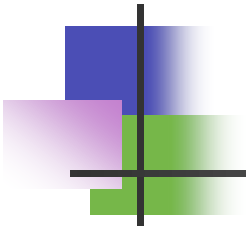
- ViewResolver
  - Seleciona uma view baseando-se em um nome lógico
- View
  - Responsável por retornar a resposta ao usuário
  - Algumas requests podem seguir para a view diretamente, sem passar pelo modelo, outras não



Spring Boot

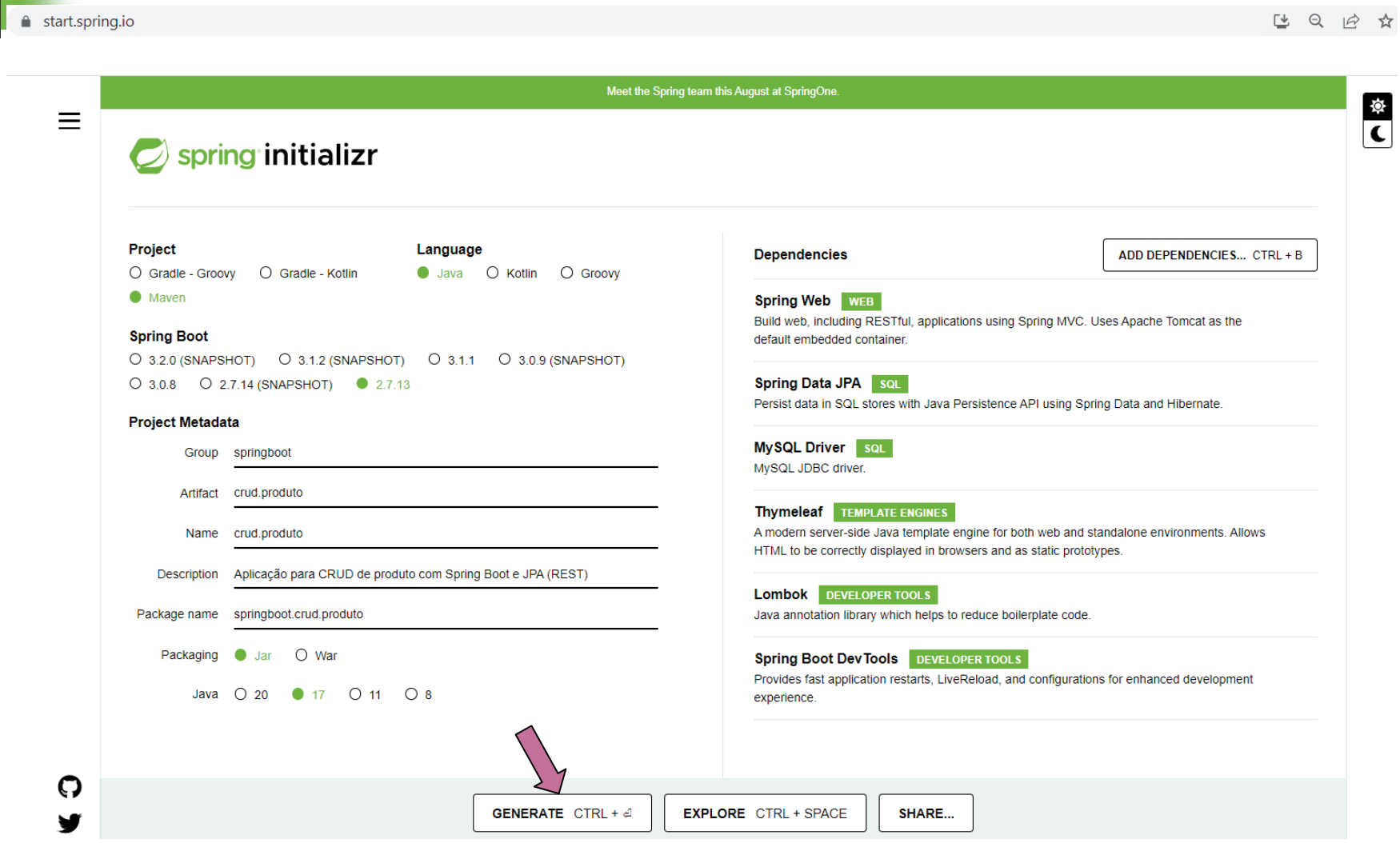
- O Spring Boot é um framework Java que facilita a criação de aplicativos independentes baseados em Spring
  - Organiza o desenvolvimento Spring com bibliotecas de terceiros
  - Integra facilmente com diversas tecnologias front e backend
  - Elimina trabalho de configuração do Spring
    - Dependências iniciais que simplificam a configuração e compilação
  - Permite criação de aplicativos Spring autônomos
  - Integra Tomcat, Jetty ou Undertow em arquivos JAR ou WAR
    - Elimina instalação desses servidores e implantação de WAR
  - Startup rápido de projetos através do Spring Initializr
    - https://start.spring.io/

Exemplo: cadastro de Produto



- Nos slides que seguem serão descritos os passos para inicialização de um projeto Spring Boot para CRUD em uma tabela produto (já feito com outras tecnologias na disciplina)
- Os fontes completos do projeto serão fornecidos em um projeto Maven Web Java

Criando o projeto start.spring.io

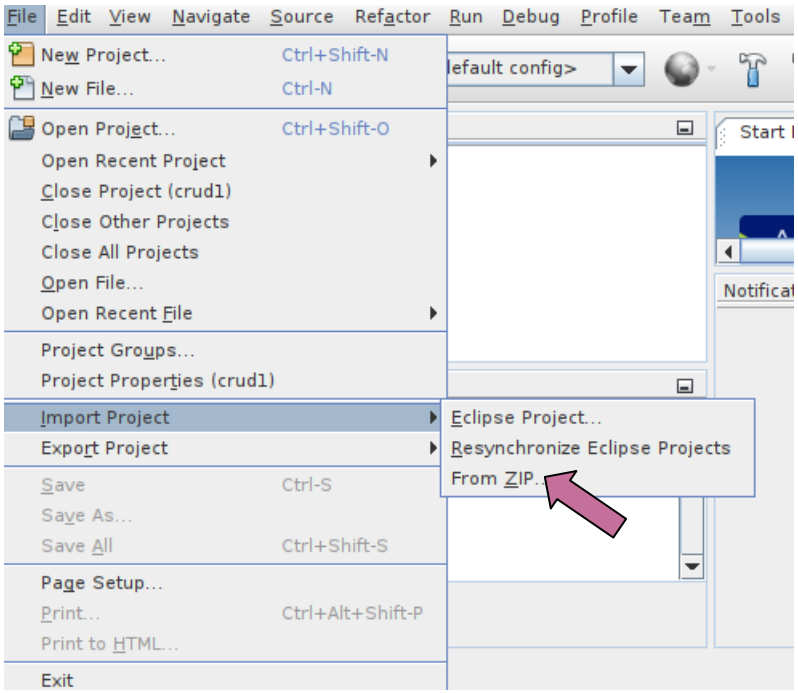


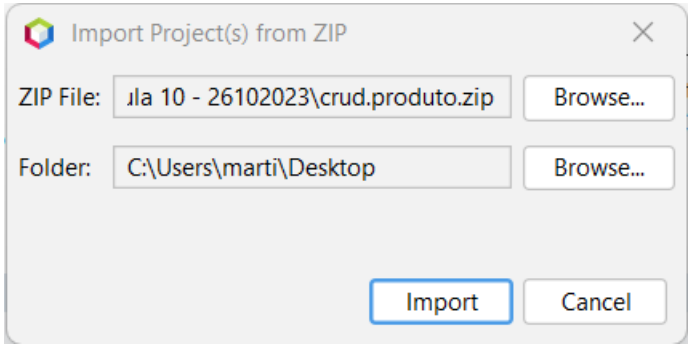
Detalhando o slide anterior

- No lado esquerdo da tela, configurações
  - Do tipo de projeto (Maven), linguagem (Java, versão 17) da versão do Spring Boot (2.7.13), tipo de empacotamento (JAR) e nome/descrição do projeto
- No lado direito da tela, as dependências
  - Spring Web: cria estrutura de aplicação Web com RESTfull, MVC Spring e Tomcat embarcado
  - Spring Data JPA: suporte a JPA com Hibernate
  - MySQL Driver
  - Thymeleaf: facilita a criação de conteúdo HTML com templates e definição de rotas para eles
  - Lombok: biblioteca de anotações com facilidades (get/setters, construtores, entre outras)
  - DevTools: facilidades para restart/refresh de aplicação durante desenvolvimento

Importando projeto no Netbeans

- Vá em File -> Import Project -> From ZIP
- No diálogo, selecione o ZIP baixado do Initializr e a pasta onde o projeto será criado, e "Import"





Baixando as dependências

- As configurações de dependências para o Maven, feitas pelo Initializr, estarão em um arquivo **pom.xml**
  - No Linux, é necessário fornecer direito de execução ao mvnw
    - Vá até a pasta do projeto e dê o comando

```
chmod +x mvnw
```

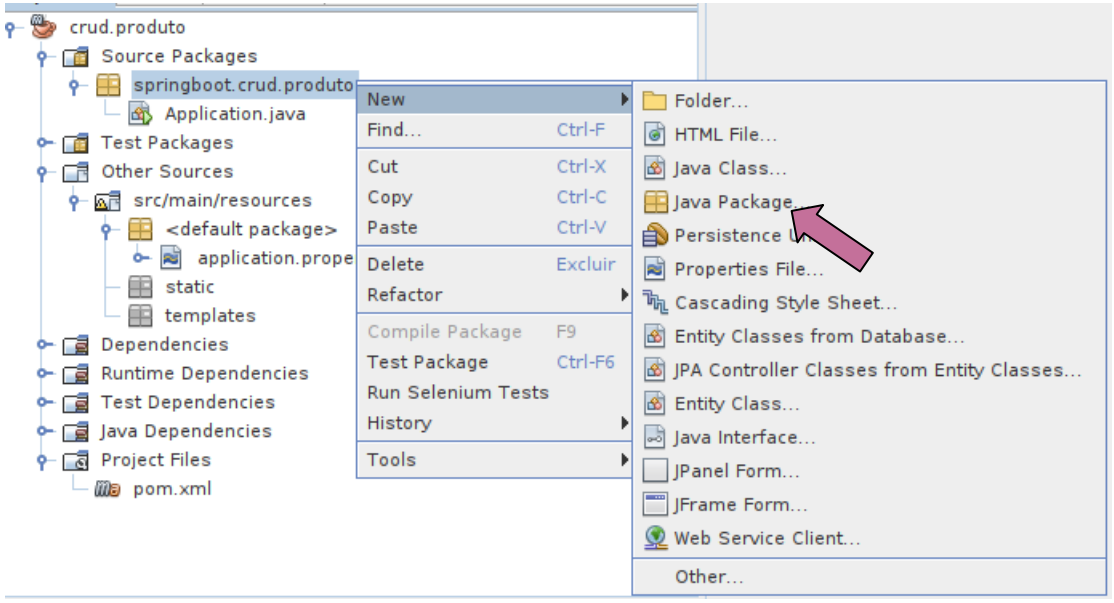
  - Configure a conexão com o DB em
    - Other Sources -> src/main/resources -> <default packages> -> application.Properties
  - Ou
    - Other Sources -> resources -> application.Properties

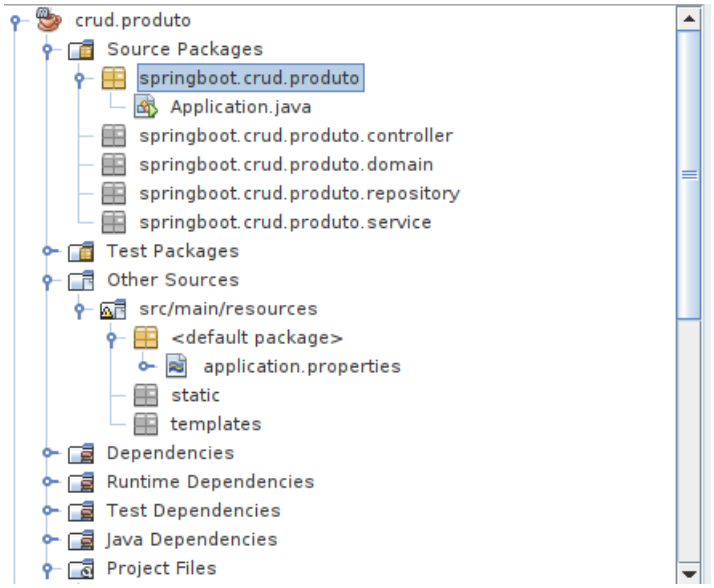
```
spring.datasource.url=jdbc:mysql://localhost:3306/curso
spring.datasource.username=aluno
spring.datasource.password=segredo
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

  - Então salve e selecione (botão direito) “Clean and Build”
    - Pode demorar alguns minutos para baixar as dependências

Criando pacotes (organizando)

- Botão direito sobre o pacote da aplicação, em Source Packages
  - Crie os pacotes (sob o pacote principal)
  - controller, domain, repository e service





Criando a classe de domínio

- Crie a classe Produto, no pacote domain

Produto.java

Source | History

```
1   package springboot.crud.produto.domain;
2
3   import ...11 lines
14
15  @Entity
16  @Getter
17  @Setter
18  @NoArgsConstructor
19  @AllArgsConstructor
20  @Table(name = "produto")
21  public class Produto implements Serializable {
22
23      @Id
24      @GeneratedValue(strategy = GenerationType.IDENTITY)
25      @Column(name = "id_produto")
26      private Integer idProduto;
27      @Column(name = "nome_produto", nullable = false)
28      private String nomeProduto;
29      @Column(name = "preco_produto", nullable = false)
30      private String precoProduto;
31      @Column(name = "quantidade_produto", nullable = false)
32      private Integer quantidadeProduto;
33
34  }
```

Criando a interface repositório

- Crie a interface ProdutoRepository, no pacote repository
  - Obs.: existem outras opções, além do JpaRepository, como o CrudRepository, que permite simplificar o controlador REST

ProdutoRepository.java

Source | History

```
1   package springboot.crud.produto.repository;
2
3   import ...4 lines
7
8   @Repository
9   public interface ProdutoRepository extends JpaRepository<Produto, Integer> {
10
11      @Query("select p from Produto p where p.id = ?1")
12      public Produto findProdutoById(Integer id);
13  }
```

Criando a classe de serviço

- Crie a classe ProdutoService, no pacote service

ProdutoService.java

Source | History

```
1   package springboot.crud.produto.service;
2
3   import ...3 lines
6
7   @Service
8   public class ProdutoService {
9
10      private final ProdutoRepository produtoRepository;
11
12      public ProdutoService(ProdutoRepository produtoRepository) {
13
14          this.produtoRepository = produtoRepository;
15      }
16
17      public Produto getProduto(Integer id) {
18          return produtoRepository.findProdutoById(id);
19      }
20  }
```

Criando a classe do controlador

- Crie a classe ProdutoController, no pacote controller

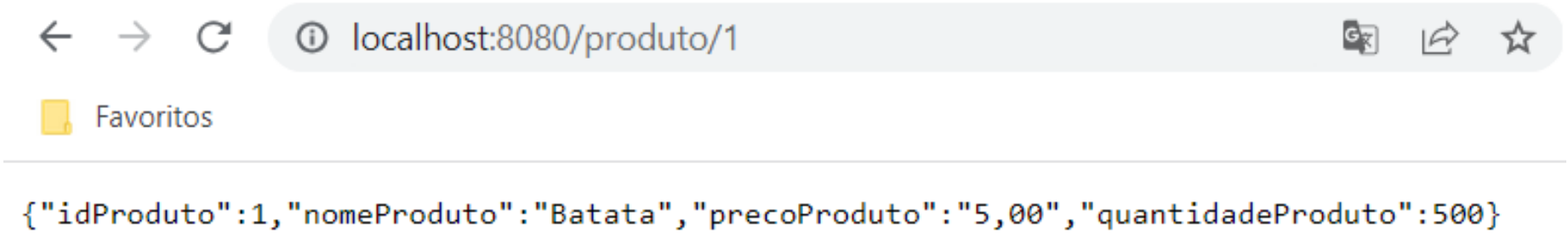
ProdutoController.java

Source | History

```
1   package springboot.crud.produto.controller;
2
3   import ...5 lines
8
9   @RestController
10  public class ProdutoController {
11
13      private ProdutoService produtoService;
14
15      public ProdutoController(ProdutoService produtoService) {
16
17          this.produtoService = produtoService;
18      }
19
20      @GetMapping(path = "/produto/{id}")
21      public Produto getProduto(@PathVariable("id") Integer id) {
22          return produtoService.getProduto(id);
23      }
    }
```

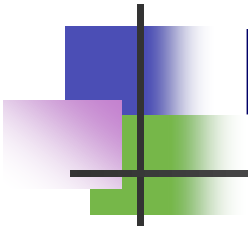
Compilando, executando e testando

- Botão direito sobre o projeto -> Properties
- Em Run, procure (browse) e selecione a classe main, Ok
- Botão direito sobre o projeto -> Clean and Build
- Botão direito sobre o projeto -> Run
- Abra um browser e teste (http://localhost:8080/produto/1)



Outras funcionalidades

- Vide projeto completo fornecido (exemplo)
  - Note que foi criado um controlador (MainController) para realizar as rotas de navegação das páginas HTML fornecidas (em templates)
  - Também note as novas funcionalidades em ProdutoController e ProdutoService
- Após executar aplicação, teste com
  - http://localhost:8080/



Bibliografia

- Material de aula Prof Marcin Sztec – Uniwersytet Łódzki – Polônia
- Dicas de estudo
  - https://www.tutorialspoint.com/spring/index.htm
  - https://www.tutorialspoint.com/spring/spring_web_mvc_framework.htm
  - https://www.bezkoder.com/spring-boot-jpa-crud-rest-api/
  - https://www.bezkoder.com/jpa-one-to-many/
  - https://blog.cod3r.com.br/java-spring-boot/
  - https://www.springboottutorial.com/spring-boot-crud-rest-service-with-jpa-hibernate
  - https://www.javainterviewpoint.com/spring-data-rest-example/
  - https://www.javaguides.net/2022/02/spring-data-jpa-one-to-many-bidirectional-mapping.html